

Học máy: Mạng Nơ-ron và Lan truyền Ngược Vector hóa

Quoc Kien

1. Bức tranh tinh thần: mạng nơ-ron đang lưu gì?

Một mạng nơ-ron không phải là một công thức bí ẩn. Nó là nhiều tầng biến đổi dữ liệu liên tiếp. Ở mỗi tầng, có hai pha:

1. **Pha tuyến tính:** gom thông tin từ tầng trước bằng trọng số và độ lệch.
2. **Pha phi tuyến:** đưa kết quả qua hàm kích hoạt để tạo giá trị kích hoạt cho tầng sau.

Với tầng l :

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$$

Ý nghĩa của từng ký hiệu:

Ký hiệu	Nằm ở đâu trong mạng?	Nói bằng lời
$\mathbf{A}^{[0]}$	Tầng đầu vào	Dữ liệu gốc, thường chính là $\mathbf{X}^T$.
$\mathbf{W}^{[l]}$	Các cạnh nối từ tầng $l - 1$ sang tầng l	Trọng số cho biết giá trị kích hoạt tầng trước ảnh hưởng mạnh yếu thế nào đến tầng sau.
$\mathbf{b}^{[l]}$	Mỗi nơ-ron ở tầng l có một độ lệch	Dịch điểm tuyến tính lên hoặc xuống trước khi kích hoạt.
$\mathbf{Z}^{[l]}$	Bên trong tầng l , trước hàm kích hoạt	Điểm tuyến tính trước kích hoạt.

Ký hiệu	Nằm ở đâu trong mạng?	Nói bằng lời
$\mathbf{A}^{[l]}$	Đầu ra của tầng l	Giá trị kích hoạt sau khi áp dụng hàm g . Đây là thứ được gửi sang tầng kế tiếp.
$d\mathbf{Z}^{[l]}$	Khi lan truyền ngược qua tầng l	Tín hiệu lỗi tại điểm trước kích hoạt.
$d\mathbf{W}^{[l]}, d\mathbf{b}^{[l]}$	Cùng chỗ với $\mathbf{W}^{[l]}, \mathbf{b}^{[l]}$	Hướng cần chỉnh trọng số và độ lệch để hàm mất mát giảm.

Nếu chỉ nhớ một câu: **lan truyền xuôi tạo ra Z rồi A ; lan truyền ngược tạo ra dZ rồi dW, db ; bước cập nhật dùng dW, db để sửa W, b .**

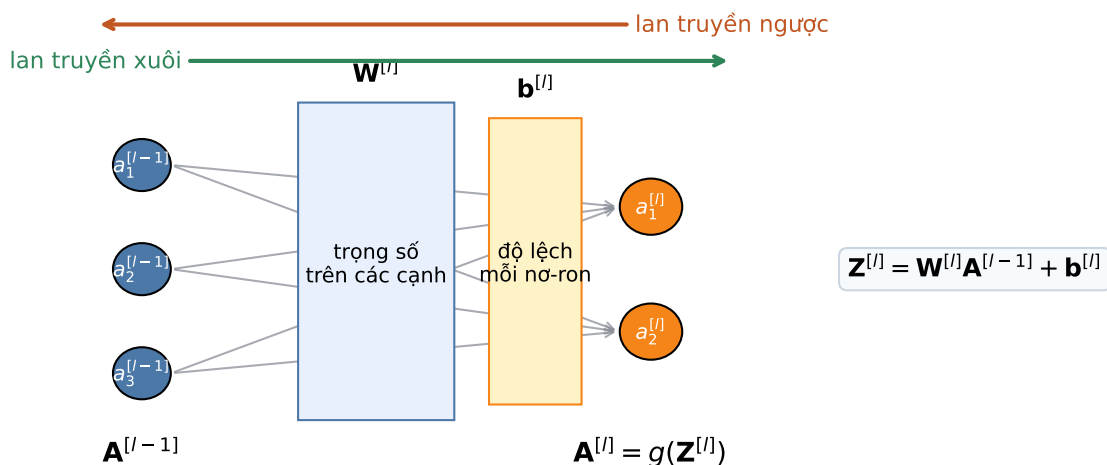


Figure 1: Vị trí của $\mathbf{A}$, $\mathbf{W}$, $\mathbf{b}$, $\mathbf{Z}$ trong một tầng kết nối đầy đủ.

2. Một nơ-ron đơn: từ đầu vào đến dự đoán

Một nơ-ron nhận vector đầu vào $\mathbf{x}$, nhân với trọng số $\mathbf{w}$, cộng độ lệch b , rồi đưa qua hàm kích hoạt:

$$z = \mathbf{w}^T \mathbf{x} + b$$

$$a = g(z)$$

Nếu g là sigmoid, a có thể đọc như xác suất. Nếu g là ReLU, a là tín hiệu dương được truyền tiếp.

Ví dụ:

$$\mathbf{x} = (2, 3), \quad \mathbf{w} = (0.5, -0.2), \quad b = 0.1$$

$$z = 0.5(2) - 0.2(3) + 0.1 = 0.5$$

Nếu dùng sigmoid:

$$a = \sigma(0.5) = \frac{1}{1 + e^{-0.5}} \approx 0.622$$

Điều quan trọng: z là điểm thô trước kích hoạt, còn a là tín hiệu đã xử lý để đưa sang bước sau.

3. Lan truyền xuôi: dữ liệu chảy từ trái sang phải

Với mini-batch có m mẫu, ta không xử lý từng mẫu riêng lẻ mà xếp chúng thành ma trận.

Nếu tầng $l - 1$ có n_{l-1} nơ-ron và tầng l có n_l nơ-ron:

Đại lượng	Kích thước	Vai trò
$\mathbf{A}^{[l-1]}$	$n_{l-1} \times m$	Giá trị kích hoạt của tầng trước cho toàn bộ mini-batch.
$\mathbf{W}^{[l]}$	$n_l \times n_{l-1}$	Mỗi hàng là trọng số đi vào một nơ-ron của tầng l .
$\mathbf{b}^{[l]}$	$n_l \times 1$	Độ lệch của từng nơ-ron, được cộng cho mọi cột trong mini-batch.
$\mathbf{Z}^{[l]}$	$n_l \times m$	Điểm tuyến tính của tầng l .
$\mathbf{A}^{[l]}$	$n_l \times m$	Đầu ra của tầng l sau hàm kích hoạt.

Lan truyền xuôi: tính dự đoán và mất mát

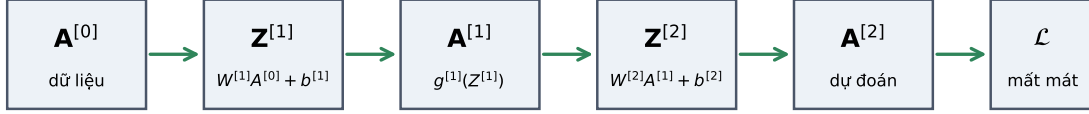


Figure 2: Lan truyền xuôi tạo Z rồi A ở từng tầng.

Tưởng tượng mỗi cột là một mẫu. Mỗi hàng là một nơ-ron. Vì vậy $\mathbf{A}^{[l]}$ có hàng là nơ-ron tầng l và cột là mẫu trong mini-batch.

4. Lan truyền ngược: lỗi chảy từ phải sang trái

Lan truyền xuôi trả lời câu hỏi: **mô hình dự đoán gì?**

Lan truyền ngược trả lời câu hỏi: **mỗi tham số phải bị sửa bao nhiêu để hàm mất mát giảm?**

Ký hiệu quan trọng nhất là $d\mathbf{Z}^{[l]}$:

$$d\mathbf{Z}^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{[l]}}$$

Đọc bằng lời: nếu điểm trước kích hoạt $\mathbf{Z}^{[l]}$ thay đổi một chút, hàm mất mát thay đổi bao nhiêu?

Sau khi có $d\mathbf{Z}^{[l]}$, ta tính được:

$$d\mathbf{W}^{[l]} = \frac{1}{m} d\mathbf{Z}^{[l]} (\mathbf{A}^{[l-1]})^T$$

$$d\mathbf{b}^{[l]} = \frac{1}{m} \sum_{i=1}^m d\mathbf{Z}^{(i)[l]}$$

Ý nghĩa trực giác:

- $\mathbf{dZ}^{[l]}$ là mức lỗi của từng nơ-ron trong từng mẫu.
- $\mathbf{A}^{[l-1]}$ là tín hiệu đã đi vào tầng đó.
- dW bằng **lỗi nhân tín hiệu đầu vào**. Nếu đầu vào lớn và lỗi lớn, cạnh đó cần được sửa nhiều.
- db là lỗi trung bình của từng nơ-ron.

Với tầng ẩn:

$$\mathbf{dZ}^{[l]} = ((\mathbf{W}^{[l+1]})^T \mathbf{dZ}^{[l+1]}) \odot g'(\mathbf{Z}^{[l]})$$

Phần $(\mathbf{W}^{[l+1]})^T \mathbf{dZ}^{[l+1]}$ kéo lỗi từ tầng sau về tầng trước. Phần $\odot g'(\mathbf{Z}^{[l]})$ kiểm tra hàm kích hoạt ở tầng này có cho gradient đi qua hay không.

Hadamard product $\odot$ là gì?

Ký hiệu $\odot$ nghĩa là nhân từng phần tử tương ứng, không phải nhân ma trận. Hai ma trận/vector phải có cùng kích thước:

$$\begin{pmatrix} 2 & -1 \\ 3 & 4 \end{pmatrix} \odot \begin{pmatrix} 5 & 6 \\ 0 & -2 \end{pmatrix} = \begin{pmatrix} 2 \cdot 5 & (-1) \cdot 6 \\ 3 \cdot 0 & 4 \cdot (-2) \end{pmatrix} = \begin{pmatrix} 10 & -6 \\ 0 & -8 \end{pmatrix}$$

Trong backprop, Hadamard product xuất hiện vì mỗi neuron ở cùng một tầng có “cổng đạo hàm activation” riêng. Ví dụ:

$$\mathbf{dZ}^{[l]} = \underbrace{(\mathbf{W}^{[l+1]})^T \mathbf{dZ}^{[l+1]}}_{\text{lỗi được kéo từ tầng sau}} \odot \underbrace{g'(\mathbf{Z}^{[l]})}_{\text{đạo hàm activation của từng neuron}}$$

Nếu neuron nào có đạo hàm activation gần 0, phần tử tương ứng trong $\mathbf{dZ}^{[l]}$ cũng bị làm nhỏ lại. Đây là lý do sigmoid bão hòa gây vanishing gradient.

Sau khi có gradient, cập nhật tham số:

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \alpha \mathbf{dW}^{[l]}$$

$$\mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \alpha \mathbf{db}^{[l]}$$

Trong đó α là learning rate.

Lan truyền ngược: hàm mất mát gửi tín hiệu sửa tham số

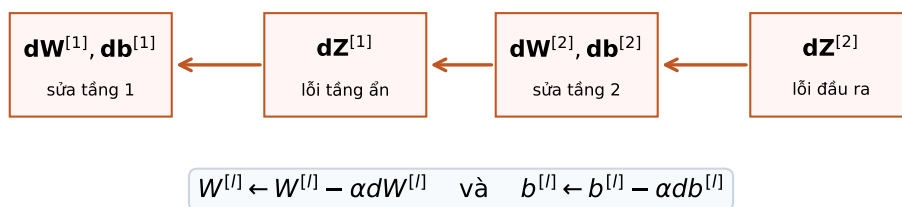


Figure 3: Lan truyền ngược đưa lỗi về từng tầng rồi tạo dW , db để cập nhật tham số.

5. Vì sao sigmoid dễ làm mất gradient?

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Đạo hàm:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Khi z rất lớn, $\sigma(z) \approx 1$, nên $\sigma'(z) \approx 0$. Khi z rất âm, $\sigma(z) \approx 0$, nên $\sigma'(z) \approx 0$. Nếu nhiều tầng đều nhân với đạo hàm gần 0, tín hiệu đạo hàm về các tầng đầu gần như biến mất. Đây là hiện tượng **gradient biến mất**.

Cận trên exam-style cho vanishing gradient.

Vì $\sigma(z) \in (0, 1)$, đặt $s = \sigma(z)$ thì:

$$\sigma'(z) = s(1 - s)$$

Hàm $s(1 - s)$ đạt cực đại tại $s = \frac{1}{2}$, nên:

$$0 < \sigma'(z) \leq \frac{1}{4}$$

Với một mạng sâu dạng vô hướng:

$$a^{[0]} = x, \quad z^{[\ell]} = w_\ell a^{[\ell-1]}, \quad a^{[\ell]} = \sigma(z^{[\ell]})$$

gradient về trọng số tầng đầu có dạng tích theo quy tắc chuỗi:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a^{[L]}} \left[\prod_{\ell=2}^L \sigma'(z^{[\ell]}) w_\ell \right] \sigma'(z^{[1]}) a^{[0]}$$

Nếu $|w_\ell| \leq 1$, phần tín hiệu đi từ tầng cuối về tầng đầu bị chặn bởi:

$$\prod_{\ell=2}^L |\sigma'(z^{[\ell]}) w_\ell| \leq \left(\frac{1}{4}\right)^{L-1}$$

Ví dụ với $L = 8$:

$$\left(\frac{1}{4}\right)^7 = \frac{1}{16384} \approx 0.000061$$

Nghĩa là chỉ riêng việc nhân chuỗi các đạo hàm sigmoid đã có thể làm tín hiệu nhỏ đi hơn 16 nghìn lần. Nếu nhiều neuron rơi vào vùng bão hòa, đạo hàm còn nhỏ hơn $\frac{1}{4}$, nên gradient biến mất nhanh hơn nữa.

ReLU:

$$g(z) = \max(0, z)$$

Đạo hàm của ReLU bằng 1 khi $z > 0$, nên ở miền dương tín hiệu đạo hàm đi qua dễ hơn.

Ba cách giảm vanishing gradient thường gặp:

- **ReLU:** tránh nhân gradient với một số luôn nhỏ hơn hoặc bằng 0.25 như sigmoid.
- **Batch Normalization:** giữ z ít rơi vào vùng bão hòa, nên đạo hàm của sigmoid/tanh ít bị kéo về 0 hơn.
- **Residual connection:** thêm đường tắt dạng $\mathbf{A}^{[l+1]} = F(\mathbf{A}^{[l]}) + \mathbf{A}^{[l]}$, giúp gradient có một nhánh identity để chảy ngược về tầng trước.

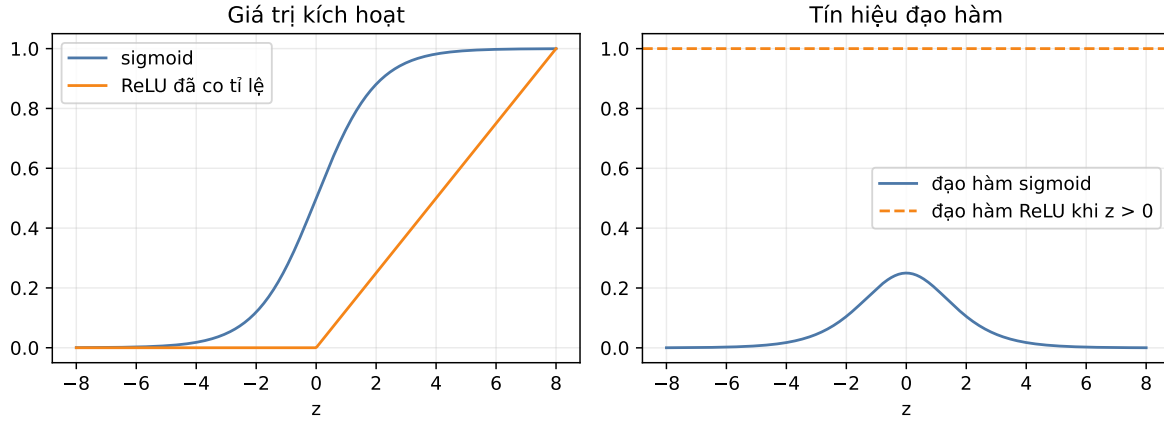


Figure 4: Sigmoid dễ bão hòa ở hai đầu, còn ReLU giữ gradient tốt ở miền dương.

6. Dropout và Chuẩn hóa Mini-batch

Dropout

Dropout tắt ngẫu nhiên một phần giá trị kích hoạt trong lúc huấn luyện:

$$\mathbf{A}_{\text{drop}}^{[l]} = \mathbf{A}^{[l]} \odot \mathbf{M}^{[l]}$$

Trong đó $\mathbf{M}^{[l]}$ là mặt nạ 0 hoặc 1. Nếu một nơ-ron bị tắt, mạng không được dựa quá nhiều vào nó. Nhờ vậy mô hình bớt overfit.

Khi kiểm thử, ta không tắt nơ-ron nữa. Với inverted dropout, lúc huấn luyện thường chia giá trị kích hoạt còn sống cho $1 - p$ để giữ kỳ vọng ổn định.

Chuẩn hóa mini-batch

Chuẩn hóa mini-batch chuẩn hóa $\mathbf{Z}^{[l]}$ trong một mini-batch:

$$\hat{z}^{(i)} = \frac{z^{(i)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

Sau đó học thêm scale và shift:

$$\tilde{z}^{(i)} = \gamma \hat{z}^{(i)} + \beta$$

Tác dụng chính:

- Giúp phân phối đầu vào của mỗi tầng ổn định hơn.
- Cho phép dùng learning rate lớn hơn.
- Thường làm quá trình train nhanh và mượt hơn.

Backward pass của Batch Normalization.

Trong bài tính tay, nếu một neuron có mini-batch $z_1, \dots, z_m$ và:

$$\tilde{z}_i = \gamma \hat{z}_i + \beta, \quad \hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

thì gradient từ tầng sau thường được ký hiệu:

$$g_i = \frac{\partial L}{\partial \tilde{z}_i}$$

Hai gradient dễ nhất là:

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^m g_i \hat{z}_i, \quad \frac{\partial L}{\partial \beta} = \sum_{i=1}^m g_i$$

Gradient quay về từng z_i có dạng gọn:

$$\frac{\partial L}{\partial z_i} = \frac{\gamma}{m\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \left[mg_i - \sum_{j=1}^m g_j - \hat{z}_i \sum_{j=1}^m g_j \hat{z}_j \right]$$

Cách nhớ: có ba phần điều chỉnh. Phần mg_i là gradient trực tiếp của phần tử i ; phần $-\sum_j g_j$ xuất hiện vì mean phụ thuộc vào cả batch; phần $-\hat{z}_i \sum_j g_j \hat{z}_j$ xuất hiện vì variance cũng phụ thuộc vào cả batch.

Ví dụ mini-batch nhanh. Với $z = (1, 3, 5, 7)$, ta có $\mu_{\mathcal{B}} = 4$, $\sigma_{\mathcal{B}}^2 = 5$, và:

$$\hat{z} = \left(-\frac{3}{\sqrt{5}}, -\frac{1}{\sqrt{5}}, \frac{1}{\sqrt{5}}, \frac{3}{\sqrt{5}} \right)$$

Nếu $\gamma = 2$, $\beta = -1$, thì:

$$\tilde{z} = 2\hat{z} - 1$$

Nếu $g = (1, -1, 2, -2)$:

$$\frac{\partial L}{\partial \beta} = 0, \quad \frac{\partial L}{\partial \gamma} = \frac{-6}{\sqrt{5}} \approx -2.683$$

Đây là đúng kiểu thay số cần cho câu BatchNorm trong đề thi.

7. Từ ký hiệu đến thao tác: đọc một bài backprop không bị lẫn

Khi làm tay một mạng nhỏ, điều dễ gây nhầm nhất là cùng một neuron có nhiều ký hiệu. Bảng sau cố định nghĩa của từng loại ký hiệu:

Ký hiệu	Nghĩa	Có phải tham số học được không?	Dùng ở đâu?
x_1, x_2	Đầu vào của mẫu dữ liệu	Không	Dùng để tính tầng ẩn.
$w_{x_1 \rightarrow h_1}$	Trọng số nối từ x_1 sang neuron ẩn h_1	Có	Cập nhật bằng gradient descent.
b_{h_1}	Bias của neuron h_1	Có	Cộng vào trước activation.
z_{h_1}	Tổng tuyến tính đi vào neuron h_1	Không	$z_{h_1} = w_{x_1 \rightarrow h_1} x_1 + w_{x_2 \rightarrow h_1} x_2 + b_{h_1}$.
h_1 hoặc a_{h_1}	Activation sau sigmoid/ReLU của neuron h_1	Không	$h_1 = \sigma(z_{h_1})$.
z_o	Tổng tuyến tính đi vào output neuron	Không	$z_o = w_{h_1 \rightarrow o} h_1 + w_{h_2 \rightarrow o} h_2 + b_o$.
$\hat{y}$	Dự đoán cuối cùng	Không	$\hat{y} = \sigma(z_o)$.
δ_o	Lỗi tại output trước activation	Không	$\delta_o = \frac{\partial L}{\partial z_o}$.
δ_{h_1}	Lỗi tại hidden neuron trước activation	Không	$\delta_{h_1} = \frac{\partial L}{\partial z_{h_1}}$.
$\frac{\partial L}{\partial w}$ hoặc dW	Gradient của loss theo trọng số	Không	Dùng để cập nhật $w \leftarrow w - \eta \frac{\partial L}{\partial w}$.

Quy tắc một dòng: z là trước activation, $a/h/\hat{y}$ là sau activation, δ là lỗi quay ngược về trước activation, còn dW/db là thứ dùng để sửa tham số.

Dịch bảng trọng số thành phương trình

Khi đề cho bảng kết nối thay vì ma trận, hãy biến từng dòng thành một cạnh trong mạng. Ví dụ một mạng 2 input, 2 hidden, 1 output có bảng:

Kết nối	Trọng số
$x_1 \rightarrow h_1$	$w_{x_1 \rightarrow h_1}$
$x_2 \rightarrow h_1$	$w_{x_2 \rightarrow h_1}$
$x_1 \rightarrow h_2$	$w_{x_1 \rightarrow h_2}$
$x_2 \rightarrow h_2$	$w_{x_2 \rightarrow h_2}$
$h_1 \rightarrow o$	$w_{h_1 \rightarrow o}$
$h_2 \rightarrow o$	$w_{h_2 \rightarrow o}$

thì phương trình forward bắt buộc là:

$$z_{h_1} = w_{x_1 \rightarrow h_1} x_1 + w_{x_2 \rightarrow h_1} x_2 + b_{h_1}, \quad h_1 = \sigma(z_{h_1})$$

$$z_{h_2} = w_{x_1 \rightarrow h_2} x_1 + w_{x_2 \rightarrow h_2} x_2 + b_{h_2}, \quad h_2 = \sigma(z_{h_2})$$

$$z_o = w_{h_1 \rightarrow o} h_1 + w_{h_2 \rightarrow o} h_2 + b_o, \quad \hat{y} = \sigma(z_o)$$

Sau đó backprop đi ngược đúng thứ tự các mũi tên:

$$o \rightarrow h_1, h_2 \rightarrow x_1, x_2$$

Nói cách khác:

1. Tính lỗi output δ_o .
2. Dùng δ_o để tính gradient của các cạnh đi vào output.
3. Truyền lỗi về từng hidden neuron để có $\delta_{h_1}, \delta_{h_2}$.
4. Dùng $\delta_{h_1}, \delta_{h_2}$ để tính gradient của các cạnh đi vào hidden.

Mẫu câu nên viết trong bài làm: “Tôi dùng z cho tổng tuyến tính trước sigmoid, dùng h cho activation sau sigmoid, và dùng δ cho đạo hàm của loss theo z tại neuron đó.” Câu này tự bảo vệ bài làm khỏi lẫn ký hiệu.

Công thức thao tác cho mạng 2-2-1, squared error

Với hidden sigmoid và output sigmoid:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Forward:

$$z_{h_1} = w_{x_1 \rightarrow h_1} x_1 + w_{x_2 \rightarrow h_1} x_2 + b_{h_1}, \quad h_1 = \sigma(z_{h_1})$$

$$z_{h_2} = w_{x_1 \rightarrow h_2} x_1 + w_{x_2 \rightarrow h_2} x_2 + b_{h_2}, \quad h_2 = \sigma(z_{h_2})$$

$$z_o = w_{h_1 \rightarrow o} h_1 + w_{h_2 \rightarrow o} h_2 + b_o, \quad \hat{y} = \sigma(z_o)$$

Backward:

$$\delta_o = (\hat{y} - y) \hat{y} (1 - \hat{y})$$

$$\frac{\partial L}{\partial w_{h_1 \rightarrow o}} = \delta_o h_1, \quad \frac{\partial L}{\partial w_{h_2 \rightarrow o}} = \delta_o h_2, \quad \frac{\partial L}{\partial b_o} = \delta_o$$

$$\delta_{h_1} = \delta_o w_{h_1 \rightarrow o} h_1 (1 - h_1), \quad \delta_{h_2} = \delta_o w_{h_2 \rightarrow o} h_2 (1 - h_2)$$

$$\frac{\partial L}{\partial w_{x_1 \rightarrow h_1}} = \delta_{h_1} x_1, \quad \frac{\partial L}{\partial w_{x_2 \rightarrow h_1}} = \delta_{h_1} x_2$$

$$\frac{\partial L}{\partial w_{x_1 \rightarrow h_2}} = \delta_{h_2} x_1, \quad \frac{\partial L}{\partial w_{x_2 \rightarrow h_2}} = \delta_{h_2} x_2$$

Update:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}, \quad b_{new} = b_{old} - \eta \frac{\partial L}{\partial b}$$

8. Quy trình debug một bài neural network

Trước khi tính dài, kiểm tra theo thứ tự:

1. Vẽ cấu trúc tầng: input, hidden, output.
2. Ghi rõ mỗi cạnh là trọng số nào và mỗi node có bias nào.
3. Tính toàn bộ z trước, rồi mới tính activation. Đừng trộn z_h với h .
4. Backprop luôn bắt đầu từ output:

$$\delta_o = \frac{\partial L}{\partial z_o}$$

5. Gradient của một trọng số luôn bằng:

lỗi tại neuron nhận $\times$ activation của neuron gửi

Ví dụ:

$$\frac{\partial L}{\partial w_{h_1 \rightarrow o}} = \delta_o h_1, \quad \frac{\partial L}{\partial w_{x_1 \rightarrow h_1}} = \delta_{h_1} x_1$$

Bẫy thường gặp: δ_h phải nhân thêm đạo hàm activation của hidden neuron. Nếu quên $h(1-h)$ với sigmoid, gradient hidden sẽ quá lớn và sai bản chất.

9. Bảng Công thức Thi: Lan truyền Xuôi, Lan truyền Ngược, Cập nhật

Giai đoạn	Công thức	Kích thước
Đầu vào	$\mathbf{A}^{[0]} = \mathbf{X}^T$	$n_0 \times m$
Tuyến tính xuôi	$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$	$n_l \times m$
Kích hoạt xuôi	$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$	$n_l \times m$
Lỗi đầu ra, sigmoid + BCE	$\mathbf{dZ}^{[L]} = \mathbf{A}^{[L]} - \mathbf{Y}$	$n_L \times m$
Lỗi tầng ẩn	$\mathbf{dZ}^{[l]} = ((\mathbf{W}^{[l+1]})^T \mathbf{dZ}^{[l+1]}) \odot g'(\mathbf{Z}^{[l]})$	$n_l \times m$
Đạo hàm theo trọng số	$\mathbf{dW}^{[l]} = \frac{1}{m} \mathbf{dZ}^{[l]} (\mathbf{A}^{[l-1]})^T$	$n_l \times n_{l-1}$
Đạo hàm theo độ lệch	$\mathbf{db}^{[l]} = \frac{1}{m} \sum_i \mathbf{dZ}^{(i)[l]}$	$n_l \times 1$
Cập nhật trọng số	$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \alpha \mathbf{dW}^{[l]}$	giống $\mathbf{W}^{[l]}$
Cập nhật độ lệch	$\mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \alpha \mathbf{db}^{[l]}$	giống $\mathbf{b}^{[l]}$

Quy tắc kiểm tra nhanh:

- dW phải cùng kích thước với W .
- db phải cùng kích thước với b .
- dZ phải cùng kích thước với Z .
- Nếu phép nhân ma trận không khớp kích thước, hãy kiểm tra lại chiều mini-batch: thường mỗi cột là một mẫu.

```
dW từ backprop: [[-0.045927  0.072171]]
dW từ sai phân số: [[-0.045927  0.072171]]
Sai lệch tuyệt đối: [[0. 0.]]
```

10. Bộ bài tập mẫu có lời giải

Bài 1: Xác định kích thước ma trận

Đề bài. Một mini-batch có $m = 5$ mẫu, tầng trước có $n_{l-1} = 3$ nơ-ron, tầng hiện tại có $n_l = 4$ nơ-ron. Hãy xác định kích thước của $\mathbf{A}^{[l-1]}$, $\mathbf{W}^{[l]}$, $\mathbf{b}^{[l]}$, $\mathbf{Z}^{[l]}$, $\mathbf{dW}^{[l]}$.

Lời giải.

Vì mỗi cột là một mẫu:

$$\mathbf{A}^{[l-1]} \in \mathbb{R}^{3 \times 5}$$

Vì tầng hiện tại có 4 nơ-ron, mỗi nơ-ron nhận 3 giá trị kích hoạt từ tầng trước:

$$\mathbf{W}^{[l]} \in \mathbb{R}^{4 \times 3}$$

Độ lệch có một số cho mỗi nơ-ron:

$$\mathbf{b}^{[l]} \in \mathbb{R}^{4 \times 1}$$

Tuyến tính xuôi:

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

Kích thước:

$$(4 \times 3)(3 \times 5) + (4 \times 1) = 4 \times 5$$

Nên:

$$\mathbf{Z}^{[l]} \in \mathbb{R}^{4 \times 5}$$

Đạo hàm theo trọng số luôn cùng kích thước với trọng số:

$$d\mathbf{W}^{[l]} \in \mathbb{R}^{4 \times 3}$$

Kết luận. Nếu dW không cùng kích thước với W , phép nhân trong backprop đã sai.

Bài 2: Lan truyền xuôi qua mạng 2-2-1

Đề bài. Cho mạng có 2 đầu vào, 2 nơ-ron ẩn dùng ReLU, và 1 đầu ra dùng sigmoid:

$$\mathbf{x} = \begin{pmatrix} 1.0 \\ 2.0 \end{pmatrix}, \quad \mathbf{W}^{[1]} = \begin{pmatrix} 0.5 & -0.4 \\ 0.3 & 0.8 \end{pmatrix}, \quad \mathbf{b}^{[1]} = \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}$$

$$\mathbf{W}^{[2]} = \begin{pmatrix} 1.2 & -0.7 \end{pmatrix}, \quad b^{[2]} = 0.05$$

Tính $\mathbf{Z}^{[1]}$, $\mathbf{A}^{[1]}$, $Z^{[2]}$, và $A^{[2]}$.

Lời giải.

Tầng ẩn:

$$\mathbf{Z}^{[1]} = \begin{pmatrix} 0.5 & -0.4 \\ 0.3 & 0.8 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}$$

Nơ-ron ẩn thứ nhất:

$$z_1^{[1]} = 0.5(1) - 0.4(2) + 0.1 = -0.2$$

Nơ-ron ẩn thứ hai:

$$z_2^{[1]} = 0.3(1) + 0.8(2) - 0.2 = 1.7$$

Do đó:

$$\mathbf{Z}^{[1]} = \begin{pmatrix} -0.2 \\ 1.7 \end{pmatrix}$$

Qua ReLU:

$$\mathbf{A}^{[1]} = \begin{pmatrix} \max(0, -0.2) \\ \max(0, 1.7) \end{pmatrix} = \begin{pmatrix} 0 \\ 1.7 \end{pmatrix}$$

Tầng đầu ra:

$$Z^{[2]} = \begin{pmatrix} 1.2 & -0.7 \end{pmatrix} \begin{pmatrix} 0 \\ 1.7 \end{pmatrix} + 0.05$$

$$Z^{[2]} = 1.2(0) - 0.7(1.7) + 0.05 = -1.14$$

Qua sigmoid:

$$A^{[2]} = \sigma(-1.14) = \frac{1}{1 + e^{1.14}} \approx 0.242$$

Kết luận. Mạng dự đoán xác suất khoảng 0.242 cho lớp 1.

Bài 3: Một bước lan truyền ngược ở tầng đầu ra

Đề bài. Với đầu ra của Bài 2 và nhãn thật $y = 1$, dùng sigmoid + binary cross-entropy. Tính $dZ^{[2]}, dW^{[2]}, db^{[2]}$.

Lời giải.

Với sigmoid + binary cross-entropy:

$$dZ^{[2]} = A^{[2]} - y$$

Thay số:

$$dZ^{[2]} = 0.242 - 1 = -0.758$$

Vì chỉ có một mẫu:

$$dW^{[2]} = dZ^{[2]}(A^{[1]})^T$$

$$dW^{[2]} = -0.758 \begin{pmatrix} 0 & 1.7 \end{pmatrix} = \begin{pmatrix} 0 & -1.289 \end{pmatrix}$$

Đạo hàm theo độ lệch:

$$db^{[2]} = dZ^{[2]} = -0.758$$

Nếu tốc độ học $\alpha = 0.1$, cập nhật tầng đầu ra:

$$W_1^{[2]} \leftarrow 1.2 - 0.1(0) = 1.2$$

$$W_2^{[2]} \leftarrow -0.7 - 0.1(-1.289) = -0.571$$

$$b^{[2]} \leftarrow 0.05 - 0.1(-0.758) = 0.126$$

Kết luận. Vì mô hình dự đoán quá thấp so với nhãn 1, đạo hàm làm độ lệch tầng đầu ra tăng và làm trọng số nối từ giá trị kích hoạt 1.7 bớt âm để lần sau xác suất lớp 1 cao hơn.

Bài 4: Backprop đầy đủ qua hidden layer với ký hiệu rõ ràng

Đề bài. Cho mạng 2-2-1, hidden sigmoid, output sigmoid, squared error:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Một mẫu dữ liệu:

$$x_1 = 2, \quad x_2 = -1, \quad y = 0$$

Tham số ban đầu:

Kết nối hoặc bias	Giá trị
$w_{x_1 \rightarrow h_1}$	0.25
$w_{x_2 \rightarrow h_1}$	-0.15
$w_{x_1 \rightarrow h_2}$	-0.35
$w_{x_2 \rightarrow h_2}$	0.45
$w_{h_1 \rightarrow o}$	0.55
$w_{h_2 \rightarrow o}$	-0.25

Kết nối hoặc bias	Giá trị
b_{h_1}, b_{h_2}, b_o	-0.05, 0.20, -0.10

Tính forward, loss, toàn bộ gradient, rồi cập nhật với $\eta = 0.1$.

Lời giải.

Forward tầng ẩn:

$$z_{h_1} = 0.25(2) + (-0.15)(-1) - 0.05 = 0.60, \quad h_1 = \sigma(0.60) \approx 0.6457$$

$$z_{h_2} = (-0.35)(2) + 0.45(-1) + 0.20 = -0.95, \quad h_2 = \sigma(-0.95) \approx 0.2789$$

Forward output:

$$z_o = 0.55(0.6457) + (-0.25)(0.2789) - 0.10 \approx 0.1854$$

$$\hat{y} = \sigma(0.1854) \approx 0.5462$$

Loss:

$$L = \frac{1}{2}(0.5462 - 0)^2 \approx 0.1492$$

Output error:

$$\delta_o = \frac{\partial L}{\partial z_o} = (\hat{y} - y)\hat{y}(1 - \hat{y}) = (0.5462 - 0)(0.5462)(0.4538) \approx 0.1354$$

Gradient tầng output:

$$\frac{\partial L}{\partial w_{h_1 \rightarrow o}} = \delta_o h_1 = (0.1354)(0.6457) \approx 0.0874$$

$$\frac{\partial L}{\partial w_{h_2 \rightarrow o}} = \delta_o h_2 = (0.1354)(0.2789) \approx 0.0378$$

$$\frac{\partial L}{\partial b_o} = \delta_o \approx 0.1354$$

Hidden error:

$$\delta_{h_1} = \delta_o w_{h_1 \rightarrow o} h_1 (1 - h_1) = (0.1354)(0.55)(0.6457)(0.3543) \approx 0.0170$$

$$\delta_{h_2} = \delta_o w_{h_2 \rightarrow o} h_2 (1 - h_2) = (0.1354)(-0.25)(0.2789)(0.7211) \approx -0.0068$$

Gradient tầng ẩn:

$$\frac{\partial L}{\partial w_{x_1 \rightarrow h_1}} = \delta_{h_1} x_1 = (0.0170)(2) \approx 0.0341, \quad \frac{\partial L}{\partial w_{x_2 \rightarrow h_1}} = \delta_{h_1} x_2 = (0.0170)(-1) \approx -0.0170$$

$$\frac{\partial L}{\partial w_{x_1 \rightarrow h_2}} = \delta_{h_2} x_1 = (-0.0068)(2) \approx -0.0136, \quad \frac{\partial L}{\partial w_{x_2 \rightarrow h_2}} = \delta_{h_2} x_2 = (-0.0068)(-1) \approx 0.0068$$

$$\frac{\partial L}{\partial b_{h_1}} = \delta_{h_1}, \quad \frac{\partial L}{\partial b_{h_2}} = \delta_{h_2}$$

Update với $\eta = 0.1$:

Tham số	Giá trị mới
$w_{x_1 \rightarrow h_1}$	$0.25 - 0.1(0.0341) = 0.2466$
$w_{x_2 \rightarrow h_1}$	$-0.15 - 0.1(-0.0170) = -0.1483$
$w_{x_1 \rightarrow h_2}$	$-0.35 - 0.1(-0.0136) = -0.3486$
$w_{x_2 \rightarrow h_2}$	$0.45 - 0.1(0.0068) = 0.4493$
$w_{h_1 \rightarrow o}$	$0.55 - 0.1(0.0874) = 0.5413$
$w_{h_2 \rightarrow o}$	$-0.25 - 0.1(0.0378) = -0.2538$
b_{h_1}	$-0.05 - 0.1(0.0170) = -0.0517$
b_{h_2}	$0.20 - 0.1(-0.0068) = 0.2007$
b_o	$-0.10 - 0.1(0.1354) = -0.1135$

Cách tự kiểm tra dấu. Vì $y = 0$ nhưng $\hat{y} = 0.5462$ còn cao, $\delta_o > 0$. Khi cập nhật $w \leftarrow w - \eta dW$, các gradient dương bị trừ đi, giúp lần dự đoán sau có xu hướng kéo $\hat{y}$ xuống thấp hơn.